Part 2. Machine Learning

Alejandro Panigada
2003597

Algorithm Choice

1.1 Random Forest

Random forest is a supervised machine learning algorithm based on ensemble learning.

During training it operates by building an increasing number of decision trees where each tree is built on a randomized subset of the data and randomized subset of features.

Random feature selection is then combined with bagging, which is responsible to reduce model variance training multiple models, in the case of Random Forest multiple decision trees, each independent from one another and then their prediction get combined by vote in classification tasks by average on regression tasks. (Breiman, 2001)

The resulting model is more stable and less sensitive to the noise found in the training data than an individual decision tree.

Random Forest handles multiclass classification natively and that means no modification is required for using this algorithm in cases such as IDS where there are multiple attack categories.

An important strength of Random Forest is its resistance to irrelevant features.

Each tree uses only a specific subset of training data allowing noise to predominate predictions and any influence is also removed by the averaging performed.

A very important strength is the built-in feature importance measuring by using mean decrease of impurity across all features. However, the use of impurity-based feature importance tends to inflate importance of classes with more support. (Louppe et al., 2013)

This can be fixed by using permutation-based feature importance instead in the case of imbalanced datasets such as IDS. (Louppe et al., 2013)

Permutation-based feature selection measures how much the F1 score drops during shuffling across samples which helps in turn measuring predictive capabilities rather than just split frequency.

Random Forest tends to be memory intensive due to storing of a large number of trees.

Additionally, Random Forest has no ability to capture temporal or sequential patterns inside a network flow.

This makes specific attacks undetectable when there are temporal patterns such as in the case of port scanning.

Also, without preprocessing the algorithm cannot handle categorical values not present during training.

This is limiting for the current dataset, since test set has attack categories not present in the training dataset. This can only be handled during preprocessing.

1.2 XGBoost

XGBoost, standing for Extreme Gradient Boosting, is a gradient boosting decision tree (GBDT) machine learning library. (Chen and Guestrin, 2016)

Providing parallel tree boosting, it is a library which is known for its good performance on regression, classification and ranking problems especially on tabular and structured data. (Chen and Guestrin, 2016)

Unlike Random Forest, which uses bagging, XGBoost is based on the concept of boosting.

Boosting consists in improving accuracy of the model by training each model sequentially. Each model created bases its prediction based on the error made by the previous one. This reduces bias and is very important for misclassification. (Friedman, 2001)

In gradient boosting, used by GBDTs, the process of additively generating weak models is achieved by the usage of a gradient descent algorithm, hence the name. (Friedman, 2001)

The sequential error corrections are also important for imbalanced datasets such as IDS sets. Without this kind of error correction, minority classes would be more easily overlooked.

It natively handles missing values and it supports early stopping to prevent overfitting when using validation data. (Chen and Guestrin, 2016)

Early stopping capability allows for monitoring of the performance on a validation set during training and the stopping of the boosting process when performance stops increasing, automatically preventing overfitting.

While it outperforms Random Forest in tabular data, it is slower in training speed. It has also more hyperparameters than Random Forest making their tuning a risk for overfitting.

The model is in general more prone to overfitting than Random Forest.

The provided datasets are also small in size and aggressive hyperparameter search can lead to overfitting easily.

Like Random Forest there is no ability in handling sequential or temporal correlation among the records.

1.3 Comparative Summary

Both algorithms are strong performers on tabular data such as IDS data and both can handle multiclass classification, crucial for labelling several types of attacks. They both handle class imbalance well. (Moustafa and Slay, 2016)

While XGBoost outperforms generally Random Forest in tabular data, training is slower due to the Boosting method. Random Forest can be memory intensive and performance stops increasing substantially after a certain number of trees is reached.

XGBoost generally should achieve higher performance on tabular data due to its decreased risk for bias. The issue is that the training set of only 100 samples has risk associated with its larger hyperparameter space, and Random Forest's simpler configuration may produce more reliable results in practice. The relative performance of the two models on this specific dataset can be determined only by evaluating on the test set.

Data Pipeline

2.1 Data Ingestion

Before any preprocessing takes place, dataset is to be inspected for both training and test dataset.

Training data consists of 100 rows and 45 columns, while test data consists of 80 rows and 45 columns.

The dataset comprises 10 possible types of traffic: Normal, Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode and Worms. (Moustafa and Slay, 2015)

Provided datasets present a highly imbalanced scenario. Normal and Generic traffic have the majority of records while other categories are represented by very few samples such as in the case of Shellcode or Worms. This imbalance directly implies that evaluation metrics must be chosen to address this imbalance. (Moustafa and Slay, 2016)

The data is divided already into training and test data, so no merge or split is to be done.

Further merging would allow statistical properties from test split to influence training. This directly means data leakage. (Raschka, 2018)

The "service" column has a "-" sign when service is unknown.

No NaN or infinite values are present.

2.2 Preprocessing

First, the "id" column is to be dropped. The column is an identifier for each record and is not usable for training as it does not bear any relationships between the rows.

If included, the model would risk during training an association of the "id" feature with attack categories leading to an inflation without truly being able to generalise.

Since "attack_cat" is the target for multiclass classification, "label" which can be used only for binary classification can be dropped.

The "attack_cat" column is kept as the target variable across both the training and test sets.

In the "service" column, each "-" sign must be replaced with "unknown".

This must be done rather than simply dropping the column as otherwise there would be significant data loss. Also replacing values with a statistical imputation would be inappropriate as the absence of a known value is not random. (Raschka, 2018)

Using "unknown" as replacement helps instead to preserve information regarding when no service was detected helping increasing the accuracy rather than penalising.

"attack_cat" is to be label encoded alongside "proto", "service" and "state".

The encoder is to be fit on the training data only. Categories present in the test set but missing from the training set are to be mapped to a reserved value to prevent runtime errors when evaluating. (Raschka, 2018)

Scaling is not necessary for tree-based algorithms.

2.3 Modelling

Both Random Forest and XGBoost are to be trained on the pre-processed training dataset.

Due to highly imbalanced datasets, both models need tuning to avoid inflation for high cardinality classes.

To do so, Random Forest must use "class_weight" set to balanced to avoid class imbalance.

XGBoost can do the same through the sample_weight hyperparameter.

Hyperparameter tuning is only to be performed in training.

Poorly chosen hyperparameters create the risk of producing a model that overfits.

Random Forest primary parameters are the number of trees and the maximum tree depth. XGBoost instead has learning rate, maximum depth and number of estimators as the most consequential. Tuning must be done conservatively using cross-validation on the training set only. (Raschka, 2018) Generalisation is more important than training performance.

2.4 Analysis

Both models are to be evaluated on the final pre-processed test dataset.

A classification report is to be generated for every model, providing per-class precision, recall and F1 score.

Auxiliary generation of a confusion matrix for each model can help identifying misclassification in both models.

Macro Average F1 score and per-class F1 score are the main metrics to be considered due to the highly imbalanced nature of the dataset. Using just accuracy or weighted averaging would allow domination by the majority classes and would mask poor performance. (Sokolova and Lapalme, 2009)

2.5 Results

Confusion matrix is the main visual element to be presented as visual benchmark due to high clarity for multiclass classification.

Macro and per-class F1 scores are the main and most reliable metrics for imbalanced data and are to be used rather than accuracy. A model which just predicts every traffic to be legitimate, by using accuracy would still achieve a high accuracy because of the majority class being benign traffic. (Sokolova and Lapalme, 2009)

Evaluation Metrics

3.1 Macro Averaged F1 Score

The F1 score is a metric which unifies precision and recall into a single value. This is achieved by computing the harmonic mean. (Sokolova and Lapalme, 2009) A classifier that reports high precision but rarely detects real attacks or one that detects all attacks but reports too many false alarms are both undesirable. F1 score means that both precision and recall have to be high for it to be high.

In multiclass classification, the F1 score is computed for each class and then each score for each class is combined as a single value. Macro-averaging combines each F1 score of each attack category and then performs averaging with equal weight. (Sokolova and Lapalme, 2009)

Because of the high class imbalance in the IDS dataset Macro Averaging is the preferable option.

Weighted average is to avoid as it gives more importance to highly frequent classes, doing the opposite of what Macro Average does.

It would produce misleading results with high scores even when the model fails to detect rare classes. (Sokolova and Lapalme, 2009)

3.2 Confusion Matrix with Recall and Precision-Recall Curve

Macro F1 does not reveal the structure of model error. The usage of a confusion matrix is preferable for checking this.

It displays, for every pair of classes, the number of times one category was predicted as another. This produces a matrix where the diagonal entries represent correct predictions.Anything else not found in the diagonal represents specific misclassification patterns.

Per-class recall indicates what proportion of actual attacks for each class the model can detect. Having a low recall on DoS attacks means that while DoS attacks are present, they are classified as Normal traffic. Ultimately it passes undetected, meaning a false negative with potentially severe consequences.

The precision-recall curve plots the trade-off of precision and recall across all classification thresholds for all classes. a large area under the precision-recall curve for each class means it is detecting true attacks with a low amount of false alarms. This is important when dealing with minority attack categories. Single threshold-based metrics may give a misleading picture of model behaviour. Per-class recall and precision-recall curves provide an overall report of how the model is behaving. This means how the model would behave and what kinds of mistakes it makes in a real-world production environment as a real IDS.

Reference List

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.

Davis, J. and Goadrich, M. (2006) 'The relationship between precision-recall and ROC curves', Proceedings of the 23rd International Conference on Machine Learning, pp. 233–240.

Friedman, J.H. (2001) 'Greedy function approximation: A gradient boosting machine', Annals of Statistics, 29(5), pp. 1189–1232.

Louppe, G., Wehenkel, L., Sutera, A. and Geurts, P. (2013) 'Understanding variable importances in forests of randomized trees', Advances in Neural Information Processing Systems, 26, pp. 431–439.

Moustafa, N. and Slay, J. (2015) 'UNSW-NB15: A comprehensive data set for network intrusion detection systems', Proceedings of the Military Communications and Information Systems Conference (MilCIS), pp. 1–6.

Moustafa, N. and Slay, J. (2016) 'The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 dataset', Journal of Information Security and Applications, 31, pp. 18–31.

Raschka, S. (2018) 'Model evaluation, model selection, and algorithm selection in machine learning', arXiv preprint arXiv:1811.12808.

Sokolova, M. and Lapalme, G. (2009) 'A systematic analysis of performance measures for classification tasks', Information Processing and Management, 45(4), pp. 427–437.